

Uso Conjunto de Duas Linguagens de Programação

Fractal de Mandelbrot: interface em Python, cálculo em C, integração via FFI

Aplicação gráfica	Conjunto de Mandelbrot, interativo (zoom e pan)
Linguagem da interface	Python 3 com Tkinter (biblioteca padrão)
Linguagem do cálculo	C (C99) compilada como biblioteca compartilhada, com pthreads
Método de integração	FFI — Foreign Function Interface — pelo módulo ctypes
Dependências externas	Nenhuma (nem NumPy, nem Pillow)

1. A aplicação escolhida

O conjunto de Mandelbrot é o conjunto dos números complexos c para os quais a sequência definida por $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Na prática, itera-se cada ponto até que $|z|$ ultrapasse um raio de escape ou até um limite máximo de iterações; a contagem de iterações até o escape define a cor do pixel.

A escolha é conveniente para o propósito do trabalho por três razões. Primeiro, o algoritmo é **trivialmente paralelizável**: cada pixel é independente dos demais, o que permite explorar threads no lado C sem qualquer sincronização. Segundo, o custo é **concentrado e mensurável** — um laço aritmético apertado, exatamente o cenário em que uma linguagem compilada se distancia de uma interpretada. Terceiro, a aplicação é **naturalmente interativa**: o zoom exige recalcular a imagem a cada clique, tornando a latência do cálculo perceptível ao usuário e a qualidade da integração diretamente observável.

Conforme o enunciado, a aplicação gráfica em si não é o objeto de apreciação; ela é o pretexto para o problema real, tratado nas seções 3 e 4: fazer duas ferramentas de vocações distintas cooperarem.

2. Divisão de responsabilidades entre as linguagens

A fronteira entre as linguagens foi traçada de modo que cada uma execute aquilo em que é boa, sem sobreposição. Não há uma única iteração do fractal escrita em Python, nem uma única linha de interface gráfica escrita em C.

Arquivo	Ling.	Responsabilidade
src/mandelbrot.h	C	Define a ABI pública — o contrato entre as duas linguagens. É o único arquivo que ambos os lados precisam conhecer.
src/mandelbrot.c	C	Serviço de cálculo. Iteração de escape, coloração contínua, testes analíticos do cardioide e do bulbo, particionamento entre pthreads.
gui/mandelbrot_ffi.py	Python	A ponte. Carrega a biblioteca, declara os protótipos, converte tipos e gerencia o buffer de pixels.
gui/main.py	Python	Interface com o usuário. Janela Tkinter, eventos de mouse e teclado, zoom/pan, exibição da imagem, gravação em disco.
gui/headless.py	Python	Cliente alternativo sem GUI, para ambientes sem servidor gráfico, e benchmark de escalabilidade.

2.1 Por que esta divisão e não outra

A justificativa é quantitativa. O mesmo kernel de escape foi implementado em Python puro e medido contra a versão em C, ambos em uma única thread e sobre a mesma região do plano:

Implementação (200×150 px, 200 iterações)	Tempo	Relação
Python puro (laço interpretado)	210 ms	1×
C, compilado com -O3, 1 thread	4,1 ms	≈ 51× mais rápido

A diferença de aproximadamente cinquenta vezes existe *antes* de qualquer paralelismo, e decorre da natureza das ferramentas: cada iteração em Python envolve despacho dinâmico de tipos e alocação de objetos float, enquanto em C a mesma operação é um punhado de instruções de ponto flutuante em registradores. Com as pthreads, o ganho escala adicionalmente com o número de núcleos.

A recíproca também vale: construir a mesma interface — janela, tratamento de eventos, diálogo de gravação — em C exigiria GTK ou Win32 e um volume de código desproporcional, enquanto em Python cabe em algumas dezenas de linhas com a biblioteca padrão. Cada linguagem foi empregada em sua vocação.

3. O método de interface entre as duas linguagens

Esta é a seção central do trabalho. A integração é feita por **FFI (Foreign Function Interface)** através do módulo `ctypes`, da biblioteca padrão do Python.

3.1 Alternativas consideradas

Mecanismo	Avaliação
<code>ctypes</code> (adotado)	Na biblioteca padrão; nenhum código de cola em C; a <code>.so</code> gerada é uma biblioteca C comum, reutilizável por qualquer linguagem com FFI. Custo por chamada é maior, mas irrelevante aqui — poucas chamadas de longa duração.
Python/C API	Máximo desempenho, mas obriga a escrever código de cola acoplado ao interpretador (PyObject, contagem de referências), e o resultado só serve ao Python.
Cython / pybind11 / SWIG	Geram o binding automaticamente, porém adicionam dependência de ferramenta e uma etapa de compilação extra, obscurecendo justamente o mecanismo que este trabalho pretende explicitar.
Subprocesso / socket / arquivo	Integração por troca de dados, não por chamada de função. Custo de serialização e cópia a cada quadro, inviável para interação em tempo real.

O `ctypes` foi escolhido por ser o mecanismo que torna a fronteira *visível*: cada conversão de tipo é declarada explicitamente no código, o que serve ao propósito didático do exercício.

3.2 Como o `ctypes` funciona

O `ctypes` executa três operações:

- Carregamento.** `CDLL(caminho)` chama `dlopen()` (POSIX) ou `LoadLibrary()` (Windows), mapeando a biblioteca no espaço de endereçamento do próprio processo Python. Não há outro processo: o código C passa a executar dentro do interpretador.
- Resolução de símbolos.** As funções são localizadas pelo nome, como strings. Isso só funciona porque C não faz *name mangling* — o símbolo `mb_render` existe literalmente com esse nome na tabela da biblioteca. (Se o cálculo fosse escrito em C++, seria obrigatório envolver as declarações em `extern "C"`; o cabeçalho do projeto já prevê isso.)
- Marshalling e chamada.** Os argumentos Python são convertidos para a representação binária esperada pelo C e colocados nos registradores/pilha conforme a convenção de chamada da plataforma (System V AMD64 no Linux).

3.3 O contrato: a ABI

A fronteira foi projetada para ser a mais estreita possível. Apenas três funções são exportadas, e por elas trafegam **somente tipos primitivos**:

```
const char *mb_version(void);

int  mb_render(uint8_t *rgb, int width, int height,
               double center_x, double center_y, double scale,
               int max_iter, int n_threads);

int  mb_iterations_at(double cx, double cy, int max_iter);
```

Nenhuma struct atravessa a fronteira. Essa restrição é deliberada: compartilhar uma struct exigiria replicar seu layout de memória no lado Python com `ctypes.Structure`, incluindo o *padding* inserido pelo compilador — uma fonte clássica de erros silenciosos, pois um desalinhamento não gera erro, apenas dados corrompidos. As structs internas do C (`mb_task_t`) permanecem invisíveis ao Python.

A compilação com `-fvisibility=hidden` reforça o contrato: apenas os símbolos marcados no cabeçalho são exportados; todo o restante (funções `static` como o kernel de escape) fica oculto.

3.4 Declaração dos protótipos

Este é o passo mais crítico da integração. O `ctypes` não consegue ler o arquivo `.h`: a biblioteca compilada não carrega informação de tipos. Portanto a assinatura precisa ser **redeclarada manualmente** no lado Python:

```
_lib.mb_render.argtypes = [  
    ctypes.POINTER(ctypes.c_uint8), # rgb (buffer de saída)  
    ctypes.c_int, ctypes.c_int,      # width, height  
    ctypes.c_double, ctypes.c_double, # center_x, center_y  
    ctypes.c_double,                  # scale  
    ctypes.c_int, ctypes.c_int,       # max_iter, n_threads  
]  
_lib.mb_render.restype = ctypes.c_int
```

Omitir `argtypes` não é uma otimização de conveniência: o `ctypes` assumiria que todo argumento é `int`. Os `double` seriam então passados pelos registradores de inteiros em vez dos de ponto flutuante (XMM0–XMM7 na convenção System V), e o C leria lixo — **sem qualquer mensagem de erro**, apenas uma imagem incorreta. É a armadilha mais comum ao usar `ctypes`, e a razão pela qual a declaração explícita é o coração do módulo-ponte.

Declarar `restype = c_char_p` em `mb_version` instrui o `ctypes` a converter o `char*` retornado em bytes Python. Como a string é estática no C, ela vive por toda a execução e pode ser lida sem risco de ponteiro pendente.

3.5 Gerência de memória através da fronteira

Este é o ponto onde integrações entre linguagens costumam falhar. O C usa `malloc/free` explícitos; o Python usa contagem de referências e coletor de lixo. **Os dois sistemas não se comunicam.** Se o C alocasse o buffer da imagem e devolvesse o ponteiro, surgiria a pergunta insolúvel: quem o libera? O Python não pode chamar `free()` sobre memória que ele não conhece, e o coletor de lixo jamais recuperaria esse bloco — vazamento a cada quadro renderizado.

A solução adotada inverte a responsabilidade: **o buffer é alocado pelo Python e apenas preenchido pelo C.**

```
n_bytes = width * height * 3
buf = ctypes.create_string_buffer(n_bytes) # alocado pelo Python
ptr = ctypes.cast(buf, ctypes.POINTER(ctypes.c_uint8))

used = _lib.mb_render(ptr, width, height, ...) # C apenas ESCRIBE

return buf.raw[:n_bytes] # bytes gerenciados normalmente pelo GC
```

Assim o Python permanece dono exclusivo da memória do início ao fim. Não há vazamento nem *double free*, e o objeto é coletado normalmente quando sai de escopo. O C, por sua vez, aloca apenas seus vetores internos de threads e os libera na mesma função — nenhuma alocação sua sobrevive ao retorno.

Como contrapartida dessa escolha, o C não pode confiar no chamador: um buffer de tamanho errado causaria corrupção de heap. Por isso `mb_render()` valida defensivamente todos os argumentos e retorna `-1` em caso de inconsistência, valor que o lado Python converte em uma exceção `ValueError` — traduzindo o idioma de erros do C (códigos de retorno) para o do Python (exceções).

3.6 Concorrência: o GIL e as pthreads

O *Global Interpreter Lock* impede que dois *bytecodes* Python executem simultaneamente, o que normalmente inviabiliza paralelismo real com threads em Python. A integração contorna isso por uma propriedade do mecanismo escolhido: **o ctypes libera o GIL automaticamente antes de chamar uma função de uma CDLL e o readquire ao retornar.**

Duas consequências se somam:

- As pthreads criadas dentro de `mb_render()` executam com paralelismo real, pois o GIL não está retido durante a chamada. O paralelismo acontece integralmente abaixo da fronteira, invisível ao Python.
- A interface permanece responsiva. A GUI dispara o cálculo em uma thread Python auxiliar; como essa thread solta o GIL ao entrar no C, o laço de eventos do Tk continua girando na thread principal e a janela não congela.

Há uma restrição a respeitar: o Tk não é *thread-safe*. A imagem, portanto, não é publicada na tela pela thread de trabalho; o resultado é reagendado para a thread principal com `root.after(0, ...)`. A regra resultante é: *calcular em qualquer thread, desenhar somente na principal.*

3.7 Transporte da imagem até a tela

O C devolve um vetor de bytes RGB entrelaçado. Para exibi-lo sem recorrer a Pillow ou NumPy, o Python antepõe um cabeçalho PPM (formato P6) aos bytes — um cabeçalho de texto de três linhas seguido dos pixels crus — e entrega o resultado ao widget `PhotoImage`, que lê PPM nativamente:

```
header = f"P6\n{width} {height}\n255\n".encode("ascii")
ppm = header + rgb # nenhuma conversão dos pixels
photo = tk.PhotoImage(data=ppm)
```

A escolha do formato RGB entrelaçado na ABI não foi arbitrária: é exatamente o layout que o PPM exige, de modo que os bytes produzidos pelo C chegam à tela **sem nenhuma transformação intermediária**. O projeto assim permanece restrito à biblioteca padrão do Python.

4. Fluxo completo de uma interação

O trajeto de um clique do usuário até o pixel na tela, atravessando a fronteira duas vezes:

#	Lado	Ação
1	Python	O usuário clica no canvas. O Tk entrega o evento com as coordenadas (px, py) em pixels.
2	Python	<code>pixel_to_complex()</code> converte o pixel em um ponto do plano complexo e atualiza centro e escala da vista.
3	Python	Uma thread auxiliar é criada; <code>create_string_buffer()</code> aloca <code>width×height×3</code> bytes.
4	ponte	O ctypes converte os argumentos, libera o GIL e chama <code>mb_render()</code>.
5	C	A janela é convertida de pixels para o plano complexo; as linhas são particionadas entre N pthreads.
6	C	Cada thread itera $z \leftarrow z^2 + c$ por pixel, aplica a coloração contínua e escreve direto no buffer do Python.
7	ponte	<code>mb_render()</code> retorna; o ctypes readquire o GIL.
8	Python	Os bytes recebem o cabeçalho PPM e o resultado é reagendado para a thread principal via <code>after(0, ...)</code> .
9	Python	PhotoImage exibe a imagem; a barra de status reporta o tempo gasto dentro do C.

Note que os passos 5 e 6 desconhecem completamente a existência do Python, e os passos 1, 2, 8 e 9 desconhecem completamente a existência do C. O acoplamento está inteiramente confinado aos passos 4 e 7 — ou seja, ao arquivo `mandelbrot_ffi.py` e ao cabeçalho `mandelbrot.h`.

A aplicação também exercita uma chamada FFI de granularidade oposta: a cada movimento do mouse, `mb_iterations_at()` é invocada para um único ponto (dois `double` → um `int`) e o resultado aparece na barra inferior. O contraste entre as duas — uma chamada longa devolvendo um megabyte e uma chamada curtíssima devolvendo um inteiro — evidencia que o custo fixo por travessia da fronteira só importa quando as chamadas são muito frequentes e muito baratas.

5. Compilação e execução

A compilação usa três flags essenciais à interoperabilidade: `-fPIC`, que gera código independente de posição (obrigatório, pois a biblioteca é mapeada em endereço arbitrário no processo do interpretador); `-shared`, que produz a biblioteca em vez de um executável; e `-fvisibility=hidden`, que restringe a exportação aos símbolos da ABI.

```
gcc -std=c99 -O3 -march=native -fPIC -fvisibility=hidden \
  -c src/mandelbrot.c -o build/mandelbrot.o
gcc -shared -o build/libmandelbrot.so build/mandelbrot.o -lpthread -lm
```

Comando	Efeito
<code>make</code>	Compila a biblioteca compartilhada.
<code>make run</code>	Caso de estudo: abre a aplicação gráfica.
<code>make run-headless</code>	Caso de estudo sem servidor gráfico; gera as imagens em out/.
<code>make test</code>	Verifica a ponte: carrega a .so e valida as três funções.
<code>make bench</code>	Mede a escalabilidade com o número de threads.

Comando	Efeito
<code>make clean</code>	Remove os artefatos gerados.

O Makefile detecta o sistema operacional e ajusta o nome da biblioteca (`.so`, `.dylib` ou `.dll`) e as flags de ligação; o lado Python faz a detecção correspondente ao procurar o arquivo. O único requisito além do compilador e do Python é o Tkinter.

6. Resultados

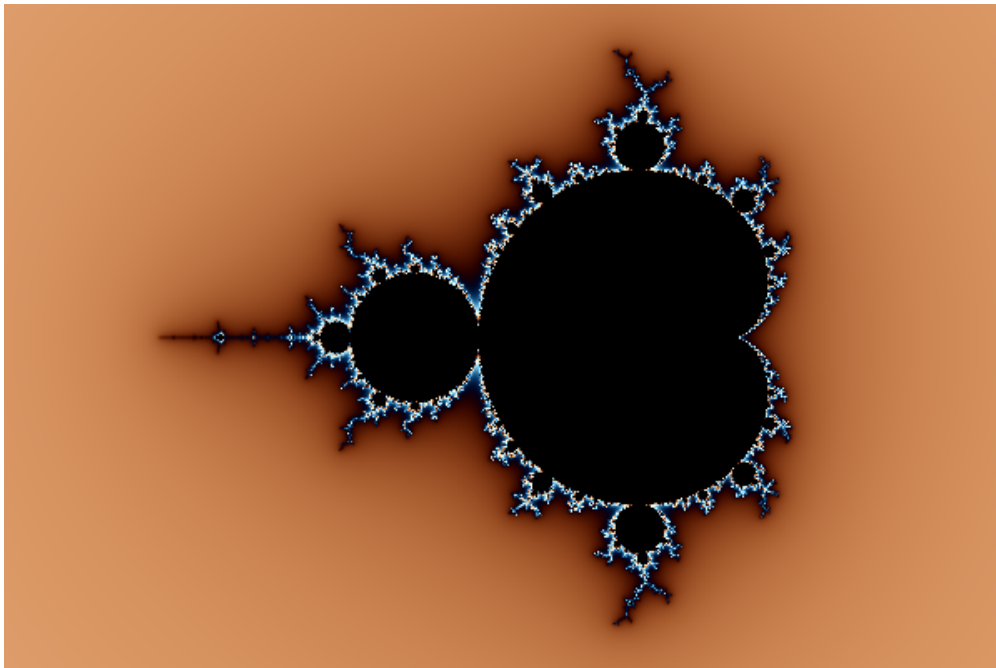


Figura 1 — Vista completa do conjunto, 1200×800, 800 iterações. Calculada em C em 186 ms e exibida pela interface em Python.

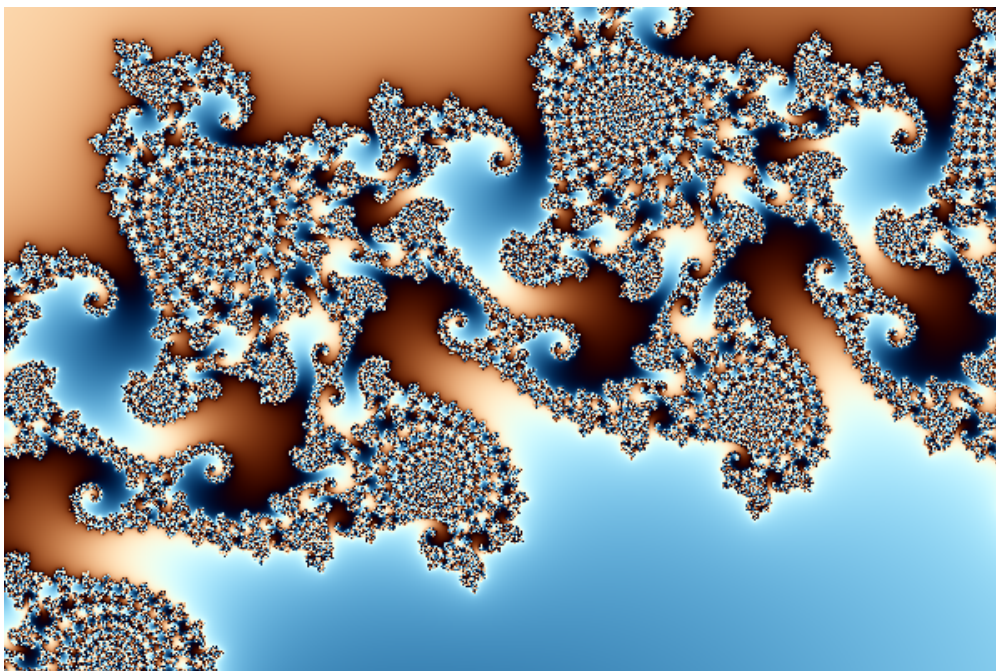


Figura 2 — Vale dos Cavalos-Marinheiros, zoom de aproximadamente 5000× em $c \approx -0,743644 + 0,131826i$, com 1500 iterações (600 ms). O aumento do custo com a profundidade do zoom é o que torna a delegação do cálculo ao C perceptível na interação.

6.1 Verificação da ponte

```
$ make test
Biblioteca C carregada de: ../build/libmandelbrot.so
Versao reportada pelo C : libmandelbrot 1.0 (C99 + pthreads)
Render 80x40 OK: 9600 bytes
Iteracoes em (0,0) : 500 (esperado: 500)
Iteracoes em (2,2) : 4 (esperado: baixo)
```

O teste valida os três aspectos da integração: a leitura da string estática confirma o marshalling de `char*`; o buffer de 9600 bytes confirma a escrita do C na memória do Python; e os pontos de controle confirmam que os `double` chegaram corretamente ao C — a origem pertence ao conjunto (atinge o teto de iterações) e o ponto (2,2) escapa em poucas iterações. Se os `argtypes` estivessem ausentes, este último teste falharia.

6.2 Escalabilidade

O `make bench` repete o cálculo variando o número de threads solicitado ao C. Em máquinas multicore o tempo cai de forma aproximadamente linear até o número de núcleos físicos, resultado esperado para um problema sem dependência entre pixels e sem sincronização — as faixas de linhas atribuídas às threads são disjuntas, dispensando qualquer mutex. No ambiente de validação utilizado, com uma única vCPU disponível, o *speedup* medido foi de 1,00× para qualquer número de threads, como seria de se esperar.

7. Considerações finais

O exercício confirma que o trabalho real de uma integração entre linguagens não está em nenhuma das duas, mas na costura entre elas. O fractal e a janela Tk são código convencional; as decisões que sustentam o projeto são todas de fronteira: manter a ABI restrita a tipos primitivos, atribuir a posse da memória a um único lado, declarar os tipos explicitamente onde o compilador não pode mais verificá-los, e escolher um formato de dados (RGB entrelaçado) que sirva aos dois lados sem conversão.

O padrão resultante — *núcleo compilado, casca interpretada* — não é particular a este projeto. É a mesma arquitetura de NumPy, TensorFlow e boa parte do ecossistema científico do Python: um kernel em C ou Fortran para o custo computacional, e uma camada em linguagem de alto nível para a expressividade e a interação. Cada ferramenta na sua vocação, com uma fronteira estreita e explícita entre elas.